
Universidad de las Fuerzas Armadas ESPE

Departamento: Ciencias de la Computación

Carrera: Ingeniería de Software

Tema: Arquitectura en 3 Capas

Taller académico N°: 3

1. Información General

- **Asignatura:** Análisis y Diseño de Software
 - **Apellidos y nombres de los estudiantes:**
Panchez Jacome Darwin Javier
Robalino Curay Cristian Isaak
Proaño Vásconez José Ignacio
 - **NRC:** 23305
 - **Fecha de realización:** 12/06/2025
-

2. Objetivo del Taller y Desarrollo

Objetivo del Taller:

Desarrollar en un entorno IDE de Java una aplicación que implemente un CRUD (Crear, Leer, Actualizar, Eliminar) para gestionar datos de estudiantes (id, nombre y edad), utilizando el patrón de arquitectura en 3 capas. Se busca demostrar cómo interactúan las capas entre sí y cómo se ejecuta la lógica del sistema desde la clase principal (Main.java).

Desarrollo:

Requisitos Funcionales

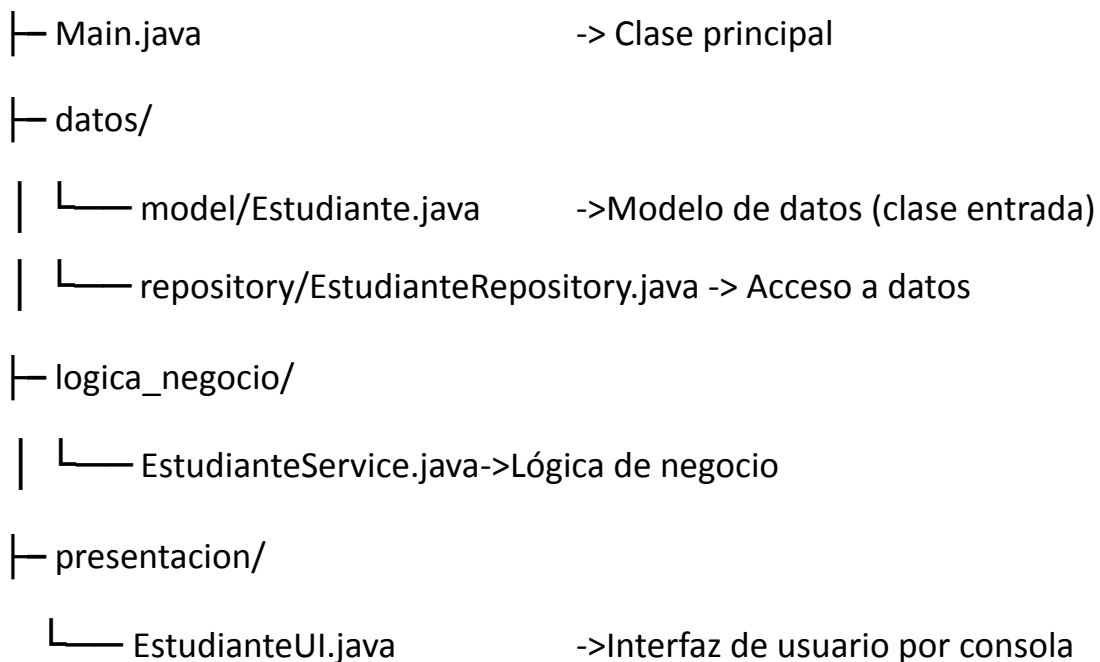
- RF1: La aplicación debe permitir agregar un nuevo estudiante con los datos: orden, nombre y edad.

- RF2: El sistema debe mostrar todos los estudiantes registrados.
- RF3: El usuario debe poder consultar un estudiante por su número de id.
- RF4: El sistema debe permitir modificar los datos de un estudiante existente.
- RF5: El sistema debe permitir eliminar un estudiante por su número de id.
- RF6: La aplicación debe estar organizada en tres capas: modelo, repositorio, servicio y

presentación.

Estructura del Proyecto

taller3Capas_analisis/



Clase Main.java

```

import datos.Repository.EstudianteRepository;
import logica_negocio.EstudianteService;
import presentacion.EstudianteUI;

/**
 * Clase principal que inicia la aplicación.

```

```

* Aquí se crean las instancias de las capas: repositorio, servicio y presentación (UI).

* La interfaz gráfica se muestra al usuario para gestionar estudiantes.

*/
public class Main {

    public static void main(String[] args) {

        // Se crea el repositorio (capa de acceso a datos)
        EstudianteRepository repo = new EstudianteRepository();

        // Se crea el servicio (lógica de negocio) usando el repositorio
        EstudianteService service = new EstudianteService(repo);

        // Se crea y muestra la interfaz gráfica, usando el servicio
        new EstudianteUI(service);

    }

}

```

Capa datos (model/estudiante.java)

```

package datos.Model;

/**
 * Clase que representa el modelo de un estudiante.
 * Contiene los atributos básicos y métodos para acceder y modificar los datos del estudiante.
 * Esta clase es utilizada en las demás capas para transferir información de los estudiantes.
 */
public class Estudiante {

    private int id;

    private String nombre;

    private int edad;

    // Constructor para inicializar los datos del estudiante
    public Estudiante(int id, String nombre, int edad) {

        this.id = id;

        this.nombre = nombre;

        this.edad = edad;
    }
}

```

```

    }

    // Métodos getter y setter para acceder y modificar los atributos

    public int getId() { return id; }

    public String getNombre() { return nombre; }

    public int getEdad() { return edad; }

    public void setNombre(String nombre) { this.nombre = nombre; }

    public void setEdad(int edad) { this.edad = edad; }

    // Método para mostrar la información del estudiante como texto
    @Override
    public String toString() {
        return "Estudiante{id=" + id + ", nombre='" + nombre + "', edad="
+ edad + '}';
    }
}

```

Capa datos (repository/EstudianteRepository.java)

```

package datos.Repository;

import datos.Model.Estudiante;
import java.util.*;

/**
 * Clase que representa el repositorio de estudiantes.
 * Se encarga de almacenar, buscar, actualizar y eliminar estudiantes
 en memoria.
 * Esta clase corresponde a la capa de acceso a datos de la aplicación.
 */
public class EstudianteRepository {

    // Mapa que almacena los estudiantes usando el id como clave

    private final Map<Integer, Estudiante> estudiantes = new HashMap<>();
}

```

```

// Agrega un estudiante al repositorio
public void agregar(Estudiante estudiante) {
    estudiantes.put(estudiante.getId(), estudiante);
}

// Busca un estudiante por su id
public Estudiante buscarPorId(int id) {
    return estudiantes.get(id);
}

// Devuelve una lista con todos los estudiantes almacenados
public List<Estudiante> obtenerTodos() {
    return new ArrayList<>(estudiantes.values());
}

// Actualiza la información de un estudiante existente
public void actualizar(Estudiante estudiante) {
    estudiantes.put(estudiante.getId(), estudiante);
}

// Elimina un estudiante por su id
public void eliminar(int id) {
    estudiantes.remove(id);
}
}

```

Capa logica_negocio (EstudianteService.java)

```

package logica_negocio;

import datos.Model.Estudiante;
import datos.Repository.EstudianteRepository;
import java.util.List;

/**

```

```
* Clase de servicio que representa la lógica de negocio para la  
gestión de estudiantes.  
  
* Aquí se validan las reglas antes de interactuar con el repositorio.  
* Esta clase conecta la capa de presentación con la de datos.  
*/  
public class EstudianteService {  
    // Referencia al repositorio de estudiantes (capa de acceso a datos)  
    private final EstudianteRepository repo;  
  
    // Constructor que recibe el repositorio a utilizar  
    public EstudianteService(EstudianteRepository repo) {  
        this.repo = repo;  
    }  
  
    // Métodos de negocio para el CRUD de estudiantes  
  
    /**  
     * Agrega un nuevo estudiante si el id no existe.  
     */  
    public void agregarEstudiante(int id, String nombre, int edad) {  
        if (repo.buscarPorId(id) != null) {  
            throw new IllegalArgumentException("El estudiante ya  
existe.");  
        }  
        repo.agregar(new Estudiante(id, nombre, edad));  
    }  
  
    /**  
     * Busca un estudiante por su id.  
     */  
    public Estudiante buscarEstudiante(int id) {  
        return repo.buscarPorId(id);  
    }  
  
    /**
```

```

    * Obtiene la lista de todos los estudiantes.
    */

    public List<Estudiante> obtenerTodos() {

        return repo.obtenerTodos();

    }

    /**
     * Actualiza los datos de un estudiante existente.
     */

    public void actualizarEstudiante(int id, String nombre, int edad) {

        Estudiante est = repo.buscarPorId(id);

        if (est == null) throw new IllegalArgumentException("No existe el estudiante.");

        est.setNombre(nombre);

        est.setEdad(edad);

        repo.actualizar(est);

    }

    /**
     * Elimina un estudiante por su id.
     */

    public void eliminarEstudiante(int id) {

        repo.eliminar(id);

    }

}

```

Capa presentacion (EstudianteUI.java)

```

package presentacion;

import logica_negocio.EstudianteService;
import datos.Model.Estudiante;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;

```

```
import java.awt.*;
import java.awt.event.ActionEvent;
import javax.swing.event.DocumentEvent;
import javax.swing.event.DocumentListener;

/**
 * Ventana principal de la aplicación CRUD de estudiantes.
 * Esta clase representa la capa de presentación usando Swing.
 * Permite al usuario agregar, mostrar, buscar, actualizar y eliminar
 * estudiantes,
 * mostrando los datos en una tabla organizada.
 */
public class EstudianteUI extends JFrame {
    // Referencia al servicio de lógica de negocio
    private final EstudianteService service;

    // Campos de texto para entrada de datos
    private final JTextField campoId = new JTextField(5);
    private final JTextField campoNombre = new JTextField(15);
    private final JTextField campoEdad = new JTextField(5);

    // Modelo y tabla para mostrar los estudiantes de forma organizada
    private final DefaultTableModel modeloTabla = new DefaultTableModel(
        new Object[]{"ID", "Nombre", "Edad"}, 0
    );

    private final JTable tabla = new JTable(modeloTabla);

    /**
     * Constructor que configura la interfaz gráfica,
     * organiza los paneles y conecta los botones con sus acciones.
     */
    public EstudianteUI(EstudianteService service) {
        this.service = service;

        setTitle("CRUD Estudiantes");

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setLayout(new BorderLayout());
    }
}
```



```
// Panel de entrada de datos

JPanel panelEntrada = new JPanel();

panelEntrada.add(new JLabel("ID:"));

panelEntrada.add(campoId);

panelEntrada.add(new JLabel("Nombre:"));

panelEntrada.add(campoNombre);

panelEntrada.add(new JLabel("Edad:"));

panelEntrada.add(campoEdad);


// Panel de botones para las operaciones CRUD

JPanel panelBotones = new JPanel();

JButton btnAgregar = new JButton("Agregar");

JButton btnMostrar = new JButton("Mostrar Todos");

JButton btnActualizar = new JButton("Actualizar");

JButton btnEliminar = new JButton("Eliminar");


panelBotones.add(btnAgregar);

panelBotones.add(btnMostrar);

panelBotones.add(btnActualizar);

panelBotones.add(btnEliminar);


// Área de tabla para mostrar los estudiantes

JScrollPane scroll = new JScrollPane(tabla);


add(panelEntrada, BorderLayout.NORTH);

add(panelBotones, BorderLayout.CENTER);

add(scroll, BorderLayout.SOUTH);


// Asignación de acciones a los botones

btnAgregar.addActionListener(this::agregar);

btnMostrar.addActionListener(_ -> mostrarTodos());

btnActualizar.addActionListener(this::actualizar);

btnEliminar.addActionListener(this::eliminar);
```

```

        // Búsqueda en tiempo real al escribir en el campo de ID
        campoId.getDocument().addDocumentListener(new DocumentListener()
        {
            public void insertUpdate(DocumentEvent e) {
                buscarEnTiempoReal();
            }

            public void removeUpdate(DocumentEvent e) {
                buscarEnTiempoReal();
            }

            public void changedUpdate(DocumentEvent e) {
                buscarEnTiempoReal();
            }
        });

        pack();

        setLocationRelativeTo(null);

        setVisible(true);
    }

    // Métodos privados para manejar cada acción CRUD y actualizar la
    // tabla

    private void agregar(ActionEvent e) {
        try {
            int id = Integer.parseInt(campoId.getText());

            String nombre = campoNombre.getText();

            int edad = Integer.parseInt(campoEdad.getText());

            service.agregarEstudiante(id, nombre, edad);

            mostrarTodos();

            JOptionPane.showMessageDialog(this, "Estudiante agregado.");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Error: " +
                ex.getMessage());
        }
    }

    private void mostrarTodos() {
        modeloTabla.setRowCount(0); // Limpiar tabla

        for (Estudiante est : service.obtenerTodos()) {

```

```

        modeloTabla.addRow(new Object[]{est.getId(), est.getNombre(),
est.getEdad()});

    }

}

// Búsqueda en tiempo real según lo que se escriba en el campo de ID
private void buscarEnTiempoReal() {

    String textoId = campoId.getText();

    if (textoId.isEmpty()) {

        mostrarTodos();

        return;

    }

    try {

        int id = Integer.parseInt(textoId);

        Estudiante est = service.buscarEstudiante(id);

        modeloTabla.setRowCount(0);

        if (est != null) {

            modeloTabla.addRow(new Object[]{est.getId(),
est.getNombre(), est.getEdad()});

        }

    } catch (NumberFormatException ex) {

        modeloTabla.setRowCount(0); // Si no es un número, limpia la
tabla

    }

}

private void actualizar(ActionEvent e) {

    try {

        int id = Integer.parseInt(campoId.getText());

        String nombre = campoNombre.getText();

        int edad = Integer.parseInt(campoEdad.getText());

        service.actualizarEstudiante(id, nombre, edad);

        mostrarTodos();

        JOptionPane.showMessageDialog(this, "Estudiante
actualizado.");

    } catch (Exception ex) {

```

```

        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage());
    }
}

private void eliminar(ActionEvent e) {
    try {
        int id = Integer.parseInt(campoId.getText());
        service.eliminarEstudiante(id);
        mostrarTodos();
        JOptionPane.showMessageDialog(this, "Estudiante eliminado.");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage());
    }
}
}

```

Características

Arquitectura en 3 Capas	
1	Cada capa cumple un rol específico (Presentación, Lógica de Negocio y Acceso a Datos), lo que mejora la organización del código.
2	Los cambios en una capa no afectan directamente a las otras, lo que permite evolucionar el sistema fácilmente.
3	Las capas pueden ser reutilizadas en otros proyectos o interfaces (por ejemplo, cambiar la interfaz gráfica sin tocar la lógica).
4	Se puede validar y controlar mejor la entrada y salida de datos al estar cada responsabilidad encapsulada en su capa.

Conclusión

Implementar una arquitectura en 3 capas permite desarrollar aplicaciones más ordenadas, mantenibles y escalables. Esta separación permite que equipos trabajen en paralelo sobre diferentes capas, y mejora la adaptabilidad del sistema. Además, cuando se requiere realizar cambios, estos pueden ejecutarse con mayor rapidez y menor riesgo, ya que se puede modificar una sola capa sin tener que rehacer todo el sistema, reduciendo significativamente el tiempo de desarrollo y adaptación frente a nuevos requerimientos.

Recomendaciones

- Utilizar buenas prácticas de programación en cada capa, como nombres claros, separación de clases, y validaciones adecuadas, para facilitar el trabajo en equipo y el mantenimiento del sistema.
- Implementar interfaces entre capas para reducir el acoplamiento, permitiendo que el sistema sea más flexible ante cambios o mejoras futuras.

3. Referencias (Norma APA 7.0)

- Descargar IntelliJ IDEA (2021, June 1). JetBrains.
<https://www.jetbrains.com/es-es/idea/download/download-thanks.html?platform=windows&code=IIC>
-